

NPTEL Week 10

ASSIGNMENT 10 SOLUTIONS & EXPLANATIONS

PROGRAMMING SOLUTIONS

1. Treasure Hunt Elimination

// PYTHON

```
n, k = [int(x) for x in input().split()]
explorers = list(range(1, n + 1))
idx = 0
while len(explorers) > 1:
    idx = (idx + k - 1) % len(explorers)
    explorers.pop(idx)
print(explorers[0], end='')

```

// CONSOLE INJECTION

```
const el = document.querySelector('textarea.inputarea, textarea');
if (el) {
    el.focus();
    document.execCommand('selectAll', false, null);
    document.execCommand('insertText', false, `n, k = [int(x) for x in input().split()]
explorers = list(range(1, n + 1))
idx = 0
while len(explorers) > 1:
    idx = (idx + k - 1) % len(explorers)
    explorers.pop(idx)
print(explorers[0], end='')`);
}

```

2. Temperature Sensor Grouping

// PYTHON

```
# Read the sensor readings (space-separated) and keep only valid ones (0-255)
readings = [int(x) for x in input().split() if 0 <= int(x) <= 255]

# Each group covers 32 values, so integer division by 32 gives the group (0-7)
groups = [value // 32 for value in readings]

# Add up all the group numbers and print the total
total = sum(groups)
print(total, end='')

```

// CONSOLE INJECTION

```
const el = document.querySelector('textarea.inputarea, textarea');
if (el) {
    el.focus();
    document.execCommand('selectAll', false, null);
    document.execCommand('insertText', false,
        `# Read the sensor readings (space-separated) and keep only valid ones (0-255)
readings = [int(x) for x in input().split() if 0 <= int(x) <= 255]

# Each group covers 32 values, so integer division by 32 gives the group (0-7)
groups = [value // 32 for value in readings]

# Add up all the group numbers and print the total
total = sum(groups)
print(total, end='')`);
}
```

3. Warehouse Product Code Matching

// PYTHON

```
a, b = [list(x) for x in input().split()]

# Remove each common character once from both strings
for ch in a[:]:
    if ch in b:
        a.remove(ch)
        b.remove(ch)

print(len(a) + len(b), end='')
```

// CONSOLE INJECTION

```
const el = document.querySelector('textarea.inputarea, textarea');
if (el) {
    el.focus();
    document.execCommand('selectAll', false, null);
    document.execCommand('insertText', false, `a, b = [list(x) for x in input().split()]

# Remove each common character once from both strings
for ch in a[:]:
    if ch in b:
        a.remove(ch)
        b.remove(ch)

print(len(a) + len(b), end='')`);
}
```

MCQ & MSQ SOLUTIONS WITH EXPLANATIONS

Scenario 1: FLAMES

Q01 Convert names to lowercase and remove spaces.

Explanation: `.lower()` converts every letter to lowercase and `.replace(" ", "")` strips out all spaces, so names are compared letter by letter regardless of case or spacing.

Q02 Total number of remaining letters after removing common letters.

Explanation: Line 9 computes `len(list1) + len(list2)` after Lines 5–8 have removed every common letter from both lists, so `count` holds only the leftover letters.

Q03 Line 13

Explanation: `flames.pop(index)` is the only statement that actually deletes an element. Line 12 just calculates the index and Line 14 only rearranges the list.

Q04 To ensure the index always stays within the current size of the list.

Explanation: `(count - 1) % len(flames)` wraps the count around the list like a circle, so the index is always between 0 and `len(flames) - 1` even as the list shrinks.

Q05 Selected Options: MSQ

—The loop in Lines 11–14 continues until only one letter remains in the flames list.

—The dictionary result maps each FLAMES letter to a relationship.

—`list(name1)` in Line 3 converts the first name into a list of characters.

Explanation: The loop condition `len(flames) > 1` stops at one letter, `result` is a dictionary from letters to relationships, and `list()` splits a string into characters. No step removes vowels.

Scenario 2: FLAMES 2 — Team Compatibility Analyzer

Q06 To make the comparison independent of letter case and spaces.

Explanation: Lowercasing makes 'A' and 'a' match, and removing spaces stops blanks from being counted as letters, so only the actual characters are compared.

Q07 The total number of characters remaining after common letters are removed.

Explanation: Line 9 adds the lengths of `list1` and `list2` only after the loop in Lines 5–8 has cancelled out the matching characters.

Q08 Line 13

Explanation: `flames.pop(index)` removes one element per iteration, reducing the size by 1. Line 14 (`flames[index:] + flames[:index]`) only rotates the list and keeps its size the same.

Q09 The modulo operator (%) converts the value into a valid list index.

Explanation: Any large count is reduced by `% len(flames)` to a value between 0 and `len(flames) - 1`, which simulates counting around a circle without an `IndexError`.

Q10 **Selected Options:** MSQ

—The while loop continues until only one element remains in the flames list.

—Removing Line 14 would change the sequence in which elements are eliminated.

—The value of count is computed only after common letters have been removed from both names

Explanation: The loop runs while `len(flames) > 1`. Line 14 rotates the list so counting restarts right after the removed element; without it counting would restart from the front and eliminate different elements. `count` (Line 9) is computed after Lines 5–8 remove common letters. The dictionary is only used for the final print, not for removal.

Scenario 3: Wild Life

Q11 **It creates a pixel access object to read individual pixel values.**

Explanation: `img.load()` returns a `PixelAccess` object, which lets the program read a pixel with `pixels[x, y]`. The image itself was already opened on Line 2.

Q12 **The dimensions of the image in pixels.**

Explanation: `img.size` returns a tuple (`width`, `height`), which is unpacked into the two variables.

Q13 **The Red, Green, and Blue intensity values of the current pixel.**

Explanation: `pixels[x, y]` returns an RGB tuple for the pixel at column `x` and row `y`, which is unpacked into `r`, `g` and `b`.

Q14 **To compute the average RGB intensity as a whole number.**

Explanation: `(r + g + b) // 3` performs floor division, so the average is an integer that can be compared directly with the threshold 127.

Q15 **20,000**

Explanation: Line 9 sits inside both nested loops, so it runs once per pixel: $200 \times 100 = 20,000$ times.

Scenario 4: PixelMap Image Quality Inspector

Q16 **Lines 7–8**

Explanation: The outer loop covers every `x` in `range(width)` and the inner loop every `y` in `range(height)`, so each `(x, y)` pair is visited exactly once.

Q17 **Tuple**

Explanation: For a color image, PIL returns each pixel as a tuple of channel values, such as `(90, 150, 180)`. This is why Line 9 can unpack it directly.

Q18 **140**

Explanation: `(90 + 150 + 180) // 3 = 420 // 3 = 140`.

Q19 **Dark**

Explanation: The condition is `average > 127`. 127 is not greater than 127, so the `else` branch runs and `dark` is incremented.

Q20

Selected Options: MSQ

—`img.load()` allows the program to access individual pixel values.

—The variables `bright` and `dark` are used as counters.

—The program processes pixels using their `(x, y)` coordinates.

Explanation: `img.load()` gives per-pixel access, `bright` and `dark` start at 0 and are incremented with `+= 1`, and pixels are read as `pixels[x, y]`. The nested loops run once per pixel (`width × height`), not once per row.
